
COLLABRIX

A Freelance Collaboration Platform

Bachelor of Technology in AI & Data Sciences
Parul University, Vadodara | April 2026

TEAM MEMBERS

Our Team

Boddu Venkata Sai Kowshik	2303031241591
Jagatha Uday Prakash	2303031240447
Immadi V R C Vardhan	2303031240436
Katreddi Hima Nagendra	2303031240520

PROJECT GUIDE

Under the Guidance of

Dr. Pragya Devi

Assistant Professor

Artificial Intelligence & Data Science Department
Parul University, Vadodara

Presentation Outline

- 1 Chapter 1: Introduction & Problem Statement
- 2 Chapter 2: Literature Survey
- 3 Chapter 3: System Analysis / SRS
- 4 Chapter 4: System Design
- 5 Chapter 5: Methodology
- 6 Chapter 6: Implementation
- 7 Chapter 7: Testing
- 8 Chapter 8: Conclusion & Future Work

CHAPTER 1

Introduction

Understanding the developer collaboration problem

Overview of the Developer Collaboration Problem

- Building software is rarely a solo endeavor — developers need collaborators
- Current landscape is deeply fragmented across LinkedIn, GitHub, Discord, Upwork
- No single platform combines discovery, communication, and project management
- Clients face equal challenges evaluating scattered candidate information

The Problem in Numbers

70%

Developers worked on
collaborative projects

45%

Reported difficulty
finding
suitable teammates

8.5h

Average weekly hours
on
coordination overhead

500%

Freelancing industry
growth
(2000-2014)

Why This Project Is Needed

- Personal experience: frustration forming teams for university projects and hackathons
- Professional world: freelance developers struggle to find collaborators for multi-skill projects
- Existing hiring platforms (Upwork, Toptal) charge 20% commission — not designed for peer collaboration
- Professional networks (LinkedIn) lack technical depth and development-specific features
- No platform specifically addresses peer-to-peer developer collaboration with integrated project management

Real-Life Problem: Priya's Story

Without Collabrix

- Post on LinkedIn — hundreds of irrelevant responses
- Browse Upwork — expensive, not for peers
- Ask in Discord — unstructured, hard to evaluate
- Personal network — limited skills available

With Collabrix

- Search by skills (Node.js, PostgreSQL) & region
- View profiles with ratings from past collaborations
- Send connection requests with project context
- Real-time messaging after connecting
- Rate each other post-project for reputation

Project Objectives

- Build a platform where developers can create detailed profiles showcasing skills, experience, and developer type
- Implement hierarchical region-based search (continent → sub-region → country)
- Provide skill-based matching system for discovering developers with complementary abilities
- Implement structured connection request system with personalized messages
- Build integrated real-time messaging system for connected developers

CHAPTER 1 | OBJECTIVES (CONTINUED)

Project Objectives (Continued)

- Create dual-perspective rating system where both clients and developers rate each other
- Implement role-based access control (developer vs. client) with different dashboards
- Build gamified leaderboard system ranking developers by projects and ratings
- Ensure data security through Row-Level Security (RLS) policies on all database tables
- Design mobile-first, responsive UI that works across all device sizes

Scope of the Project

Included in Current Version

- Email/password authentication
- Role selection (Developer/Client)
- Developer profiles with skills & region
- Region-based & skill-based search
- Connection requests & messaging
- Dual rating system & leaderboard
- Job browsing & notifications

Planned for Future

- Native mobile application
- Payment processing & escrow
- Video/audio calling
- AI-powered skill matching
- GitHub/GitLab integration
- Team formation automation

CHAPTER 2

Literature Survey

Analyzing existing systems and identifying gaps

CHAPTER 2 | EXISTING SYSTEMS

Upwork & Fiverr

- Upwork: \$3.8 billion in freelancer billings (2023) — largest freelance marketplace
- Fiverr: serves millions of buyers globally for project-based work
- Fundamentally designed as client-freelancer platforms, NOT peer collaboration tools
- Developers compete against each other rather than collaborating
- Significant commissions: 20% on Upwork, 20% flat on Fiverr
- No concept of team formation or developer-to-developer networking

Toptal & LinkedIn

Toptal

- Top 3% of freelance talent — highly exclusive
- Excludes students and early-career professionals
- Purely a hiring platform — no peer collaboration

LinkedIn

- 1 billion+ members — general-purpose network
- Lacks technical skill verification
- No project-based rating system

GitHub & Discord

GitHub

- 100M+ developers — code-centric platform
- No developer discovery by availability
- No structured messaging for outreach
- No project-based reputation system

Discord/Slack

- Real-time communication but unstructured
- Lack structured profiles & searchability
- No project management or accountability
- No rating system for future collaborators

Limitations of Current Systems

- No platform combines discovery + connection + messaging + project management + ratings
- Freelance platforms designed for transactions, not peer collaboration
- LinkedIn lacks technical depth and development-specific features
- GitHub excels at code but fails at human collaboration and discovery
- Informal channels (Discord) lack structure, accountability, searchability
- No region-based hierarchical filtering for compatible time zones
- Rating systems are typically one-directional (client → freelancer)

Gap in Existing Research

- Academic research focused on open-source contribution patterns (Dabbish et al., 2012)
- Studies on distributed team dynamics (Herbsleb, 2007) and pair programming (Williams, 2003)
- Very little research on initial discovery and matching — how developers find each other
- Existing algorithmic matching studies ignore geography, communication, and availability
- Notable absence of research on bilateral reputation systems in developer collaboration
- Collabrix fills this gap by combining discovery, communication, and bilateral reputation

Comparison with Proposed System

Feature	Upwork	LinkedIn	GitHub	Discord	Collabrix
Developer Profiles	Yes	Yes	Partial	No	Yes
Skill-Based Search	Yes	Partial	No	No	Yes
Region Filtering	Yes	Yes	No	No	Hierarchical
Connection Requests	No	Yes	No	No	Yes
Real-Time Messaging	Yes	Yes	No	Yes	Yes
Bilateral Ratings	No	No	No	No	Yes
Peer Collaboration	No	No	Yes	Partial	Yes
Leaderboard	No	No	Partial	No	Yes
Free for Devs	No (20%)	Freemium	Free	Free	Free

CHAPTER 3

System Analysis

Software Requirements Specification (SRS)

Users of the System

Developers

- Students, early-career, experienced freelancers
- Create profiles, discover other developers
- Frontend, Backend, Full-stack, Mobile, DevOps
- Build reputation through ratings

Clients

- Individuals or organizations hiring developers
- Post projects with skill/budget requirements
- Discover talent through search & ratings
- Dedicated client dashboard

CHAPTER 3 | FUNCTIONAL REQUIREMENTS

Functional Requirements (1/3)

FR1: User Registration & Auth

Email/password via Supabase Auth, role selection, email verification, session tokens

FR2: Profile Management

Full name, bio, avatar, skills array, region, developer type, RLS policies

FR3: Developer Search & Discovery

Hierarchical region filtering, name search, skill filtering, connection status display

Functional Requirements (2/3)

FR4: Connection Management

Send requests with optional message, accept/reject, notification badges, duplicate prevention

FR5: Real-Time Messaging

Text messages via WebSocket, persisted in DB, unread indicators, connection-scoped

FR6: Job/Project Browsing

Display available projects, search and filter by type, location, and skills

Functional Requirements (3/3)

FR7: Rating System

1-5 scale with optional review text, project_ratings and user_ratings tables, average calculation

FR8: Leaderboard

Developer ranking by completed projects and average ratings, visible to all authenticated users

Non-Functional Requirements

Performance

Page load < 3s, messages < 500ms, search < 1s, 100+ concurrent users

Security

RLS on all tables, security-definer functions, roles in separate table, Supabase Auth

Usability

Mobile-first responsive, bottom nav, toast notifications, cross-browser

Reliability

Meaningful error messages, auto-reconnect for realtime, splash screen during auth

Hardware & Software Requirements

Category	Requirement
Processor	Intel Core i5 or equivalent
RAM	8 GB or more
Storage	256 GB SSD

Software Stack

Layer	Technology	Purpose
Frontend	React 18 + TypeScript	UI and SPA routing
Build Tool	Vite 5	Fast dev server & bundling
UI Components	shadcn/ui + Radix UI	Accessible components
Styling	Tailwind CSS v3	Utility-first responsive
Backend	Supabase (PostgreSQL)	DB, auth, realtime
Authentication	Supabase Auth	Email/password auth
Real-Time	Supabase Realtime	WebSocket messaging
Routing	React Router v6	Client-side navigation
Animation	Framer Motion	Smooth UI transitions

CHAPTER 4

System Design

Architecture, data flow, and module design

Overall Architecture

Modern two-tier architecture: Client-side SPA + Serverless BaaS

Frontend (React 18 + TypeScript)

- SPA handling UI + application logic
- shadcn/ui components, Tailwind CSS
- Communicates directly with Supabase

Backend (Supabase BaaS)

- PostgreSQL DB with RLS policies
- Supabase Auth (JWT tokens)
- Realtime WebSocket channels
- Security-definer functions for role checks

Data Flow: Registration & Onboarding

- 1 User fills email/password on Auth page
- 2 Supabase Auth creates account, sends verification email
- 3 After verification, user logs in and receives session token
- 4 System checks user_roles table for existing role
- 5 If no role, user selects Developer or Client
- 6 Role inserted into user_roles table
- 7 Profile record created in profiles table
- 8 User redirected to role-specific dashboard

Data Flow: Search & Connection

- 1 Developer opens Connect screen → Find Developers tab
- 2 System queries profiles table with filters (region, skills, type)
- 3 RLS ensures only authenticated users can view profiles
- 4 Developer views profile and sends connection request
- 5 Connection stored with status 'pending' → receiver gets notification
- 6 On acceptance, messaging enabled between the two users

Module 1 & 2: Authentication & Roles

Authentication Module

- Supabase Auth: bcrypt hashing, JWT tokens
- useAuth hook wraps onAuthStateChange
- Auto session management in localStorage
- Email verification before sign-in

Role Management Module

- Separate user_roles table (security best practice)
- PostgreSQL enum: 'developer' | 'client'
- Security-definer: has_role(), get_user_role()
- useUserRole hook for conditional rendering

Module 3 & 4: Search & Connections

Developer Search Module

- Hierarchical 3-level: Continent → Sub-region → Country
- RegionFilter component with static data map
- Dynamic queries: .eq(), .ilike(), .contains()
- Cards show avatar, name, skills, connection status

Connection Management

- connections table: sender, receiver, status, message
- Duplicate prevention before sending
- Accept/reject with status update
- useNotifications hook for badge count

Module 5 & 6: Messaging & Ratings

Real-Time Messaging

- Supabase Realtime (postgres_changes)
- Connection-scoped messaging
- Chat UI with timestamps

Rating System

- Dual tables: project_ratings + user_ratings
- 1-5 stars with reviews
- RatingDialog, RatingDisplay components

Database Tables Overview

Table	Description
profiles	User profiles with skills, bio, region, avatar, developer type
user_roles	Role assignments (developer/client) - separate for security
connections	Connection requests: sender, receiver, status, message
messages	Chat messages: connection_id, sender, content, read status
projects	Job listings: title, description, skills, budget, status
project_ratings	Project-level ratings (1-5) with review text
user_ratings	User-level ratings (1-5) with review text

UI/UX Design Philosophy

- Mobile-first responsive design — optimized for smartphones
- Bottom navigation bar for intuitive mobile navigation
- Semantic color tokens with dark mode support via CSS variables
- 40+ reusable shadcn/ui components for consistency
- Toast notifications (Sonner) for user feedback
- Framer Motion animations for smooth transitions
- Role-specific dashboards with different navigation items

CHAPTER 5

Methodology

Step-by-step system workflows and algorithms

How the System Works

High-Level User Journey



After onboarding, developers browse jobs, connect with peers, and collaborate.
Clients post projects, discover talent, and manage ongoing work.

Developer Search & Matching Flow

Step 1: Filter Selection

User selects continent → sub-region → country via RegionFilter component

Step 2: Query Construction

Supabase query built with `.ilike()` for name, `.eq()` for region, `.contains()` for skills

Step 3: Results Rendering

Cards show avatar, name, bio, skills badges, developer type, connection status

Step 4: Connection Action

Connect button opens dialog for optional message, inserts into connections table

Connection & Messaging Logic

- Connection follows request-accept model (similar to LinkedIn)
- Sender and receiver IDs stored with request message and 'pending' status
- Receiver sees notification badge count via useNotifications hook
- On acceptance → status updated to 'accepted' → messaging enabled
- MessageThread uses Supabase Realtime (postgres_changes channel)
- Listens for INSERT events on messages table filtered by connection_id
- Messages stored with connection_id for scoping — no cross-connection leaks
- Real-time delivery without polling — instant via WebSocket

Rating & Reputation Algorithm

$$\text{Reputation Score} = (\text{Avg User Rating} \times 0.6) + (\text{Avg Project Rating} \times 0.3) + (\text{Completed Projects} \times 0.1)$$

Component	Weight	Source	Range
User Rating Average	60%	user_ratings table	1.0 - 5.0
Project Rating Average	30%	project_ratings table	1.0 - 5.0
Completed Projects	10%	projects table	0 - N

Weighted formula ensures personal reliability carries most weight, followed by project quality, with volume bonus.

Role-Based Access Control Flow

Database Level

- Security-definer functions bypass RLS
- `has_role()` checks `user_roles` table
- `get_user_role()` via RPC from frontend
- Prevents recursive RLS policy evaluation

Application Level

- `useUserRole` hook fetches role via RPC
- Developer: Jobs, Connect, Leaderboard, Profile
- Client: Dashboard, Find Talent, Messages, Profile
- `BottomNav` renders different tabs per role

CHAPTER 6

Implementation

Frontend structure, backend, and database schema

Frontend Structure

components/cards/

BadgeCard, ChallengeCard, LeaderboardRow, ProjectCard, RatingStatCard

components/connect/

ConnectionRequests, DeveloperSearch, MessageThread

components/layout/

BottomNav, Header — responsive navigation

hooks/

useAuth, useUserRole, useNotifications, useProjects

screens/

JobsScreen, ConnectScreen, LeaderboardScreen, ClientDashboard, etc.

integrations/supabase/

Auto-generated client and TypeScript types

Backend Structure (Supabase)

- No dedicated backend server — no Express.js, no REST API routes in code
- All data operations through Supabase JavaScript client library
- Security enforced at database level through PostgreSQL RLS policies
- Configuration in `supabase/config.toml`
- Schema changes via timestamped SQL migrations in `supabase/migrations/`
- Two critical security-definer functions:
 - `has_role(user_id, role)` — checks role in RLS policies
 - `get_user_role(user_id)` — returns role via RPC from frontend

Key API Endpoints (Supabase Operations)

Operation	Table	Description
supabase.auth.signUp()	auth.users	Register new user
supabase.auth.signInWithPassword()	auth.users	Login and get session
supabase.from('profiles').select()	profiles	Get developer profiles
supabase.from('connections').insert()	connections	Send connection request
supabase.from('messages').insert()	messages	Send a message
supabase.from('projects').select()	projects	Browse job listings
supabase.rpc('get_user_role')	user_roles	Get current user role
supabase.from('user_ratings').insert()	user_ratings	Rate a user

CHAPTER 7

Testing

Unit, integration, and end-to-end testing

Testing Strategy

- Three levels: Unit testing, Integration testing, End-to-End manual testing
- Unit: Individual components and hooks tested in isolation
- Integration: Cross-component workflows validated
- E2E: Complete user journeys tested manually
- Special attention to RLS policies and real-time subscription behavior
- Serverless architecture requires unique testing approaches

CHAPTER 7 | UNIT TESTING

Unit Testing Results

Component	Test Case	Status
useAuth hook	User not logged in → null	PASS
useAuth hook	Valid login → user object	PASS
useUserRole	Developer role assigned	PASS
RegionFilter	Select Asia → sub-regions	PASS
DeveloperSearch	Empty search → all devs	PASS
DeveloperSearch	Search by name → filtered	PASS
ConnectionRequests	Accept request → accepted	PASS
RatingDialog	Submit 4-star → saved	PASS
BottomNav (dev)	4 tabs rendered	PASS
BottomNav (client)	4 client tabs rendered	PASS

CHAPTER 7 | INTEGRATION TESTING

Integration Testing: Connection Flow

Step	Action	Expected	Status
1	Search developer by skill	Matching profiles shown	PASS
2	Send connection request	Stored as pending	PASS
3	Receiver views requests	Badge shown	PASS
4	Accept connection	Status → accepted	PASS
5	Send message	Real-time delivery	PASS
6	Receiver sees message	Visible in thread	PASS

End-to-End Test Scenarios

Scenario 1: Developer Discovery

- Aman registers as frontend developer, searches backend developers in South Asia.
- Result: Region filter works, 3 developers found, connection & messaging successful.

Scenario 2: Client Hires Developer

- Startup founder registers as client, posts project (React + Node.js), finds matching developers.
- Result: Project visible in Jobs, Find Talent shows matches, connection & messaging work.

CHAPTER 8

Conclusion & Future Work

What we achieved and what's next

What We Achieved

- Hierarchical region-based search across all continents and sub-regions
- Structured connection request system with personalized messages
- Real-time messaging via WebSocket — instant delivery without polling
- Dual rating system creating accountability from both sides
- Secure role-based access with separate user_roles table
- RLS policies on ALL tables ensuring data authorization
- Mobile-first UI with native-like experience on devices
- 40+ reusable components ensuring consistency and maintainability

Current Limitations

- Job listings use mock data — no live job API integration
- Search is filter-based, not AI-powered for smart matching
- Only text messages supported — no file sharing
- Leaderboard algorithm is simple — doesn't account for project complexity
- No payment processing or escrow system
- No email notifications — users must be in-app
- No GitHub/GitLab integration for portfolio verification
- No admin dashboard for platform management

Future Work

AI-Powered Matching

ML models for complementary skill suggestions

GitHub Integration

Automatic portfolio and skill verification

Video/Audio Calling

WebRTC-based communication

Payment Processing

Stripe with escrow and milestones

React Native App

Native mobile with push notifications

Analytics Dashboard

Profile views, acceptance rate, trends

References

- [React Documentation \(2024\)](#) — [react.dev](#)
- [TypeScript Documentation \(2024\)](#) — [typescriptlang.org](#)
- [Vite Documentation \(2024\)](#) — [vitejs.dev](#)
- [Tailwind CSS \(2024\)](#) — [tailwindcss.com](#)
- [shadcn/ui \(2024\)](#) — [ui.shadcn.com](#)
- [Supabase Documentation \(2024\)](#) — [supabase.com](#)
- [PostgreSQL RLS Policies \(2024\)](#)
- [Stack Overflow Developer Survey \(2023\)](#)
- [GitHub Octoverse Report \(2024\)](#)
- [Dabbish et al. \(2012\)](#) — Social Coding in GitHub, ACM CSCW
- [Vasilescu et al. \(2015\)](#) — CI in GitHub, ESEC/FSE
- [Williams & Kessler \(2003\)](#) — Pair Programming Illuminated

THANK YOU

COLLABRIX

A Freelance Collaboration Platform

Questions?